AWS EC2 File Deletion Approval & Audit System

PROBLEM

Accidentally deleting important files or directories on an AWS EC2 virtual machine (VM) can cause data loss and operational issues.

HOW I DEBUGGED IT

I identified the risk of executing deletion commands without approval and built a Python-based approval workflow integrated with Slack notifications.

THE SOLUTION

- Implemented approval-based file and directory deletion on AWS EC2 VMs.
- Integrated Slack notifications for deletion approval requests.
- Allowed deletion only after authorized approval.
- Cancelled rejected or expired requests without deleting protected files.
- Added audit logs to track deletion requests, approvers, decisions, and execution status.
- Tested both approval and rejection workflows to verify the expected behavior.

NOTE: The complete source code, installation steps, clear explanations, and configuration guide will be shared on the last page for easy reference.

Has anyone else faced this problem?

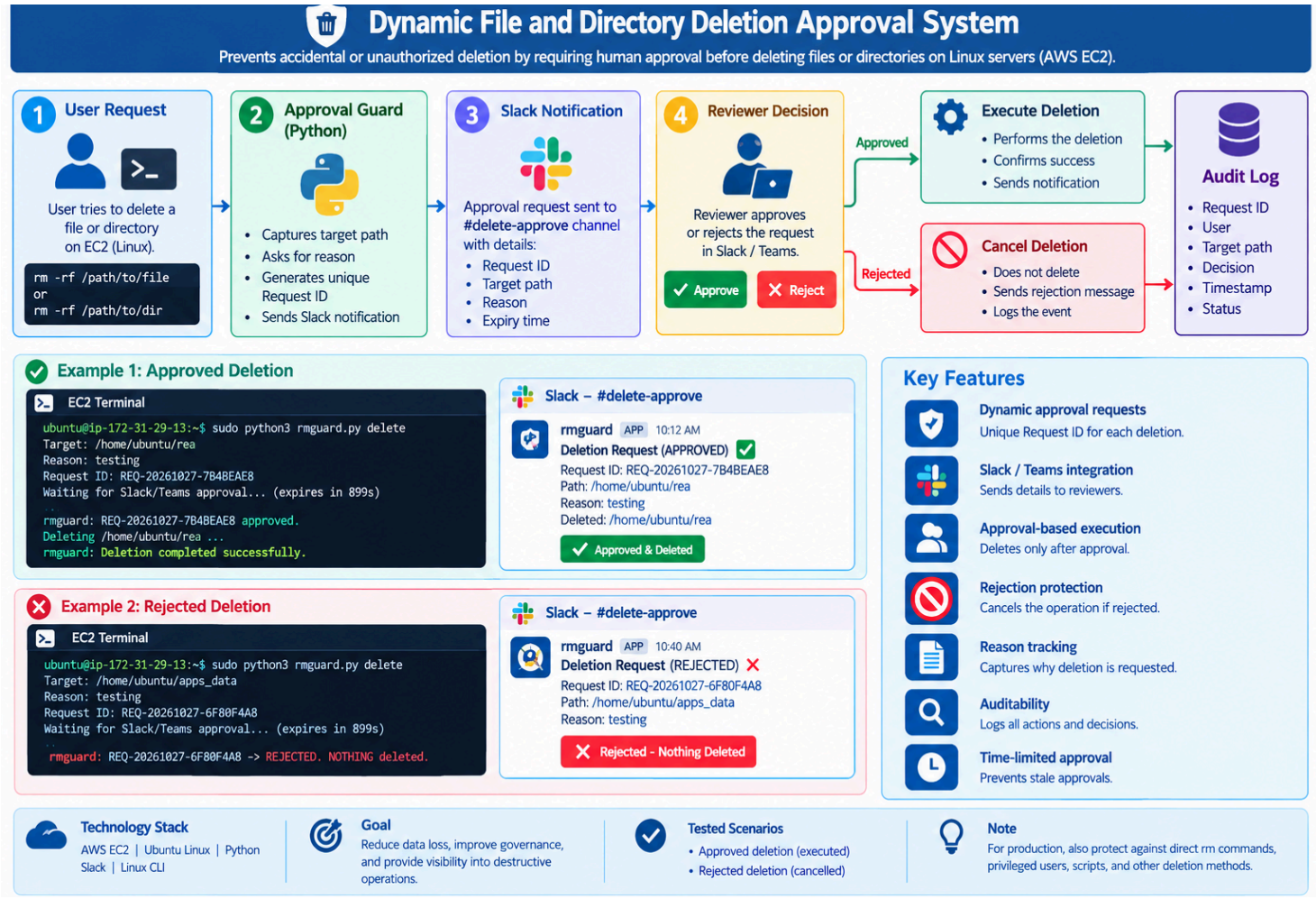
How did you fix it?

Blog:https://vishnusai.hashnode.dev/aws-ec2-file-deletionapproval-audit-system?utm_source=hashnode&utm_medium=feed

HashnodProfile:https://hashnode.com/@vishnusaiK

GitHub : https://github.com/ksaivishnusaikvs/DevOps-Engineering/tree/main/Linkedin/Post

➢ Built a Python-based file and directory deletion approval system on AWS EC2 with Slack notifications. Deletions proceed only after approval, while rejected

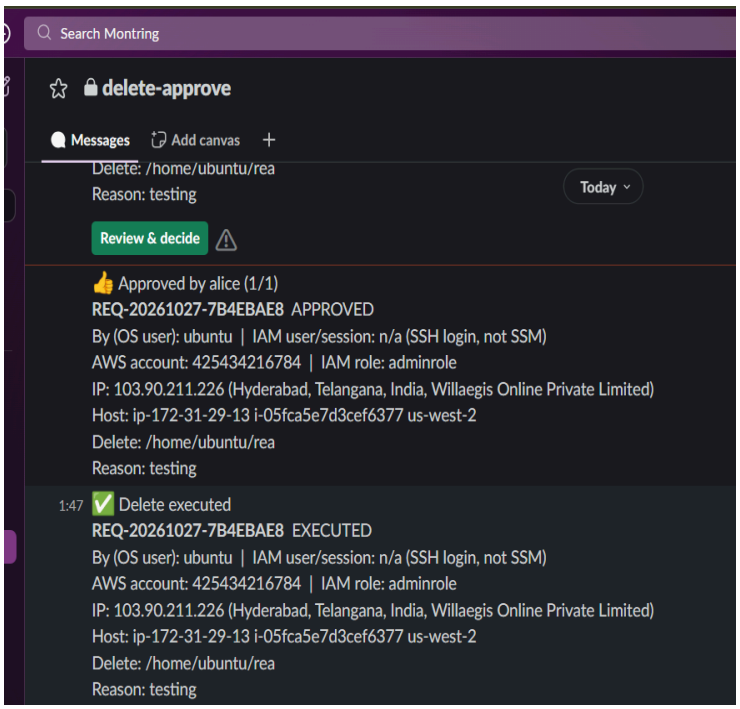



```
ubuntu@ip-172-31-29-13:~$ rm -rf rea/
Reason for deleting (shown to approvers): testing
rmguard: REQ-20261027-7B4EBAE8 waiting for Slack/Teams approval (expires in 899s)...
rmguard: REQ-20261027-7B4EBAE8 approved. /home/ubuntu/rea: deleted
ubuntu@ip-172-31-29-13:~$ ls
Audit.py          audit_run.log              bumblebee-projects-root.ndjson  deleteapprove.py   guard_demo                        v3cost.py
Fileaudit.py      auditec2.py                bumblebee-projects.ndjson       devops             guardtest                         v4cost.py
apps_data         aws-ai                     c2.py                           fileaudit.py       old                               v5cost.py
audit.py          bumblebee-devops.ndjson    cost.py                         fileaudit.py.save  rmguard.py                        v6cost.py
audit_report.docx bumblebee-ec2.ndjson       dashboard.html                  fileaudit_watch.py three-tier-app                    v8cost.py
audit_report.pdf  bumblebee-inventory.ndjson delete_approval_guard.py        go                 updated-Jarvis-by-Kashur-Engineer- watch_config.json
ubuntu@ip-172-31-29-13:~$ rm -rf apps_data/
Reason for deleting (shown to approvers): testing
rmguard: REQ-20261027-6F80F4A8 waiting for Slack/Teams approval (expires in 899s)...
rmguard: REQ-20261027-6F80F4A8 -> REJECTED. NOTHING deleted.
ubuntu@ip-172-31-29-13:~$ ls
```

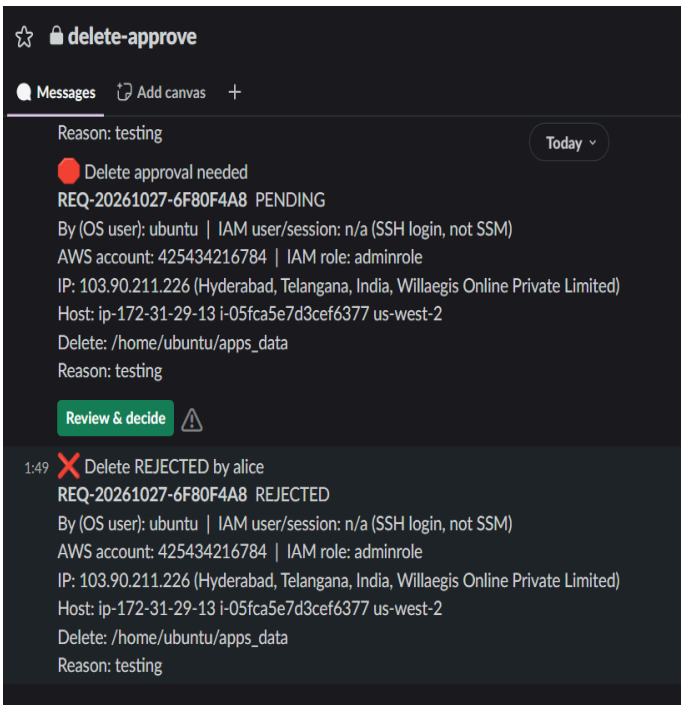
Step 1: Delete request sent to Slack/Teams for approval. Approved requests proceed with deletion; rejected requests delete nothing. Check below screen and images

```
ubuntu@ip-172-31-29-13:~$ ls
Audit.py          audit_run.log             bumblebee-projects-root.ndjson  deleteapprove.py   guard_demo      updated-Jarvis-by-Kashur-Engineer-  watch_config.json
Fileaudit.py      auditec2.py               bumblebee-projects.ndjson       devops             guardtest       v3cost.py
apps_data         aws-ai                    c2.py                           fileaudit.py       old             v4cost.py
audit.py          bumblebee-devops.ndjson   cost.py                         fileaudit.py.save  rea             v5cost.py
audit_report.docx bumblebee-ec2.ndjson      dashboard.html                  fileaudit_watch.py rmguard.py      v6cost.py
audit_report.pdf  bumblebee-inventory.ndjson delete_approval_guard.py       go                 three-tier-app  v8cost.py
ubuntu@ip-172-31-29-13:~$ rm -rf rea/
Reason for deleting (shown to approvers): testing
rmguard: REQ-20261027-7B4EBAE8 waiting for Slack/Teams approval (expires in 899s)...
rmguard: REQ-20261027-7B4EBAE8 approved. /home/ubuntu/rea: deleted
ubuntu@ip-172-31-29-13:~$ ls
Audit.py          audit_run.log             bumblebee-projects-root.ndjson  deleteapprove.py   guard_demo                          v3cost.py
Fileaudit.py      auditec2.py               bumblebee-projects.ndjson       devops             guardtest                           v4cost.py
apps_data         aws-ai                    c2.py                           fileaudit.py       old                                 v5cost.py
audit.py          bumblebee-devops.ndjson   cost.py                         fileaudit.py.save  rmguard.py                          v6cost.py
audit_report.docx bumblebee-ec2.ndjson      dashboard.html                  fileaudit_watch.py three-tier-app                      v8cost.py
audit_report.pdf  bumblebee-inventory.ndjson delete_approval_guard.py       go                 updated-Jarvis-by-Kashur-Engineer-  watch_config.json
ubuntu@ip-172-31-29-13:~$ rm -rf apps_data/
Reason for deleting (shown to approvers): testing
rmguard: REQ-20261027-6F80F4A8 waiting for Slack/Teams approval (expires in 899s)...
rmguard: REQ-20261027-6F80F4A8 -> REJECTED. NOTHING deleted.
ubuntu@ip-172-31-29-13:~$ ls
Audit.py          audit_run.log             bumblebee-projects-root.ndjson  deleteapprove.py   guard_demo      v3cost.py
Fileaudit.py      auditec2.py               bumblebee-projects.ndjson       devops             guardtest       v4cost.py
apps_data         aws-ai                    c2.py                           fileaudit.py       old             v5cost.py
audit.py          bumblebee-devops.ndjson   cost.py                         fileaudit.py.save  rmguard.py      v6cost.py
audit_report.docx bumblebee-ec2.ndjson      dashboard.html                  fileaudit_watch.py three-tier-app  v8cost.py
audit_report.pdf  bumblebee-inventory.ndjson delete_approval_guard.py       go                 updated-Jarvis-by-Kashur-Engineer-  watch_config.json
```

1Image 1 Image Approved; delete



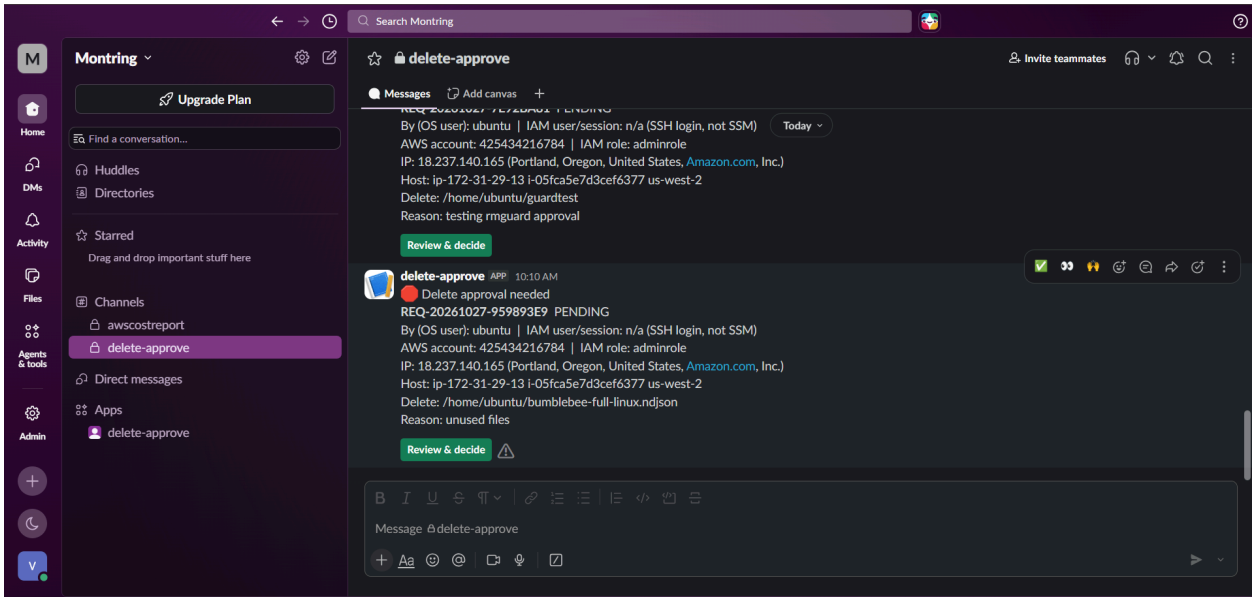
2 Image Rejected for deletion

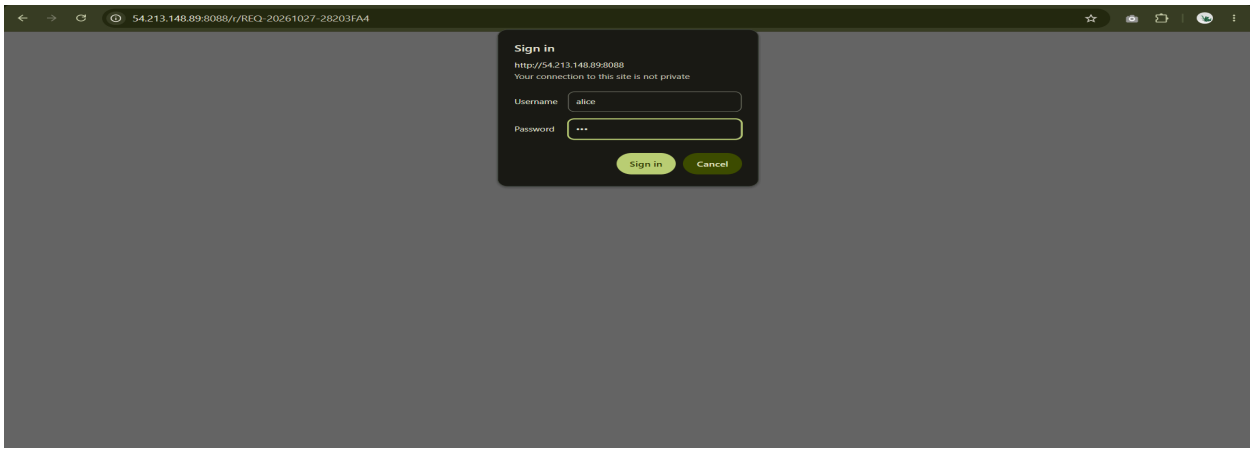


Step 2: Run the delete command. The request waits for Slack/Teams approval.

```
-rw-rw-r--  1 ubuntu ubuntu     148 Oct 10 15:35 watch_config.json
ubuntu@ip-172-31-29-13:~$ ls
Audit.py           auditec2.py                bumblebee-projects-root.ndjson  devops                  rmguard.py                          v6cost.py
Fileaudit.py       aws-ai                     bumblebee-projects.ndjson       fileaudit.py            rmguardv1.py                        v8cost.py
apps_data          bumblebee-deep.ndjson      c2.py                           fileaudit.py.save       three-tier-app                      watch_config.json
audit.py           bumblebee-devops.ndjson    cost.py                         fileaudit_watch.py      updated-Jarvis-by-Kashur-Engineer-
audit_report.docx  bumblebee-ec2.ndjson       dashboard.html                  go                      v3cost.py
audit_report.pdf   bumblebee-full-linux.ndjson delete_approval_guard.py       guard_demo              v4cost.py
audit_run.log      bumblebee-inventory.ndjson deleteapprove.py                guardtest               v5cost.py
ubuntu@ip-172-31-29-13:~$ rm bumblebee-full-linux.ndjson
Reason for deleting (shown to approvers): unused files
rmguard: REQ-20261027-959893E9 waiting for Slack/Teams approval (expires in 899s)...
```

Step 3: Slack displays the deletion request details. Click Review & decide, then select Approve Click Review & decide, sign in, user, and select Approve to delete or Reject to cancel the deletion.





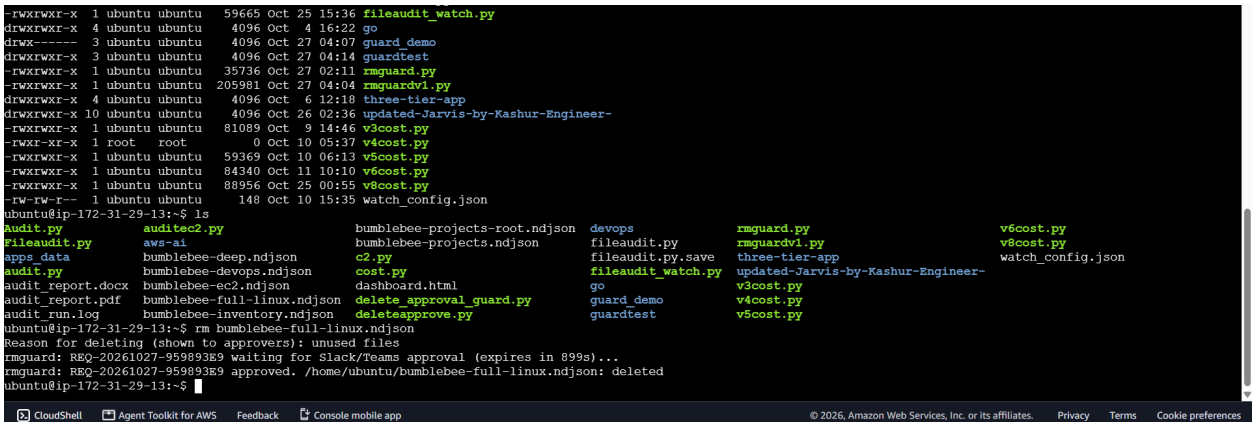
Step 4: Review the deletion request details, then click Approve to authorize deletion or Reject to cancel it.



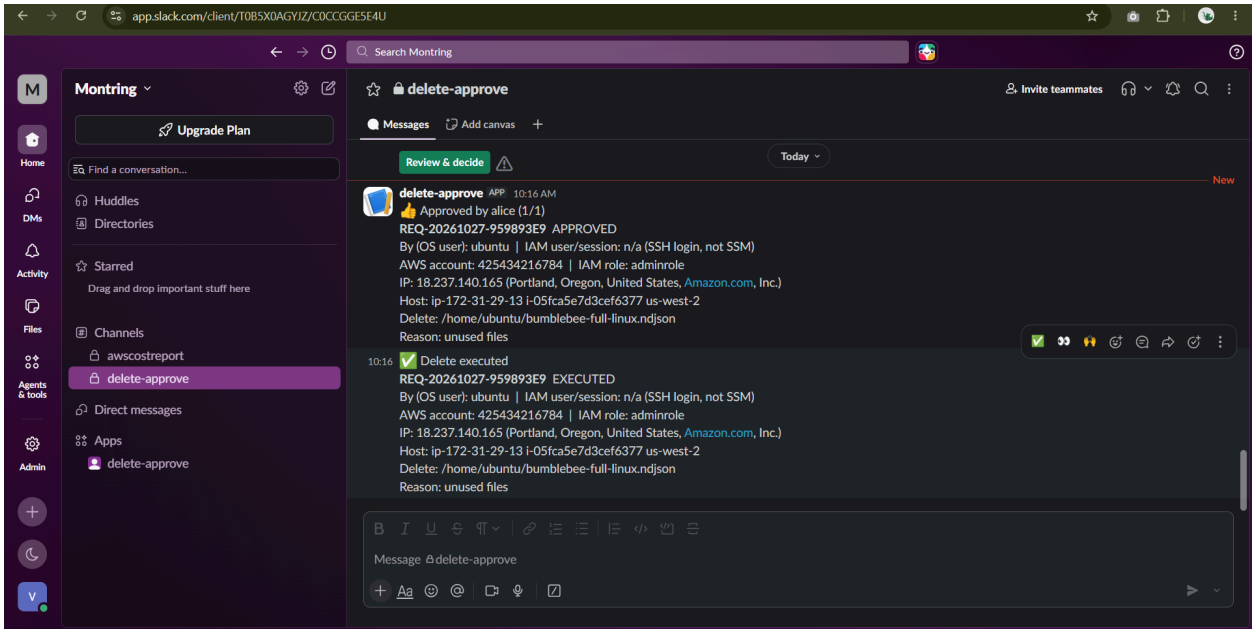
Step 5: Verify that the decision is recorded and the request status is updated to Approved or Rejected.



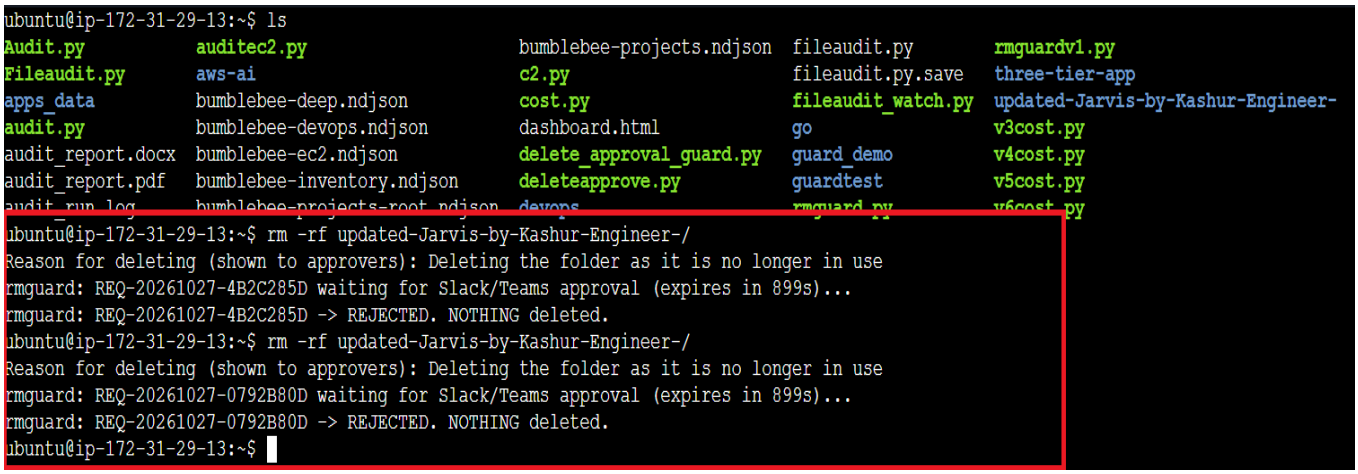
Step6: Verify in the terminal that the approved deletion was executed successfully.



Step7: Check Slack for the Approved and Delete executed confirmation messages.



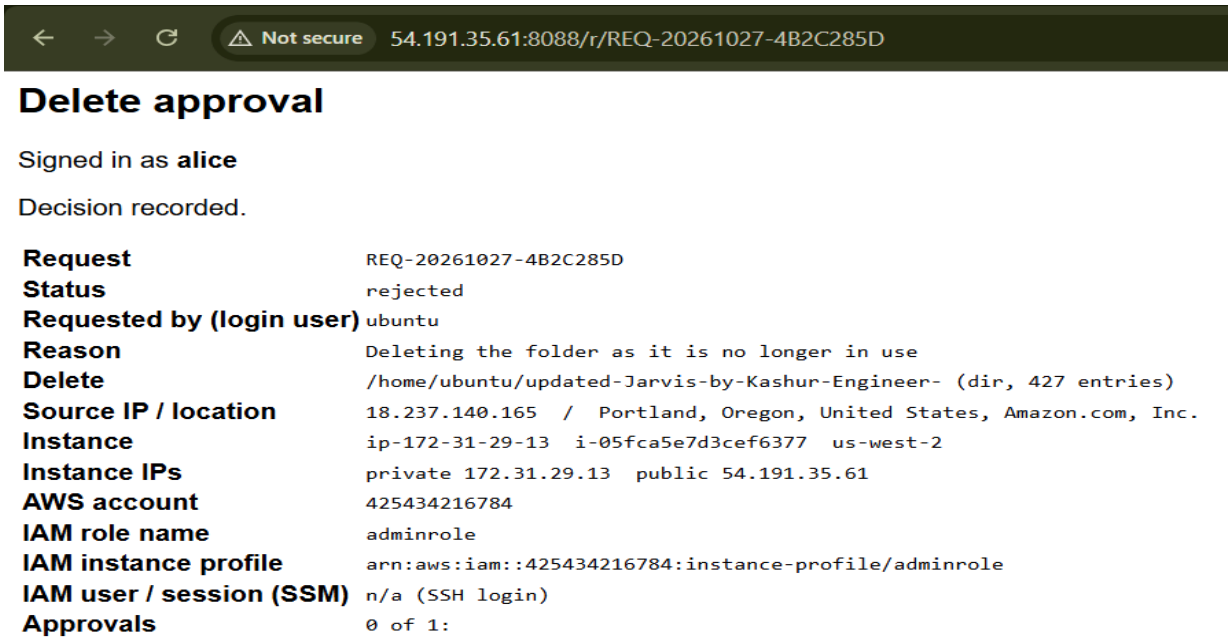
Step8: Click Reject on the approval page to test the rejection workflow.



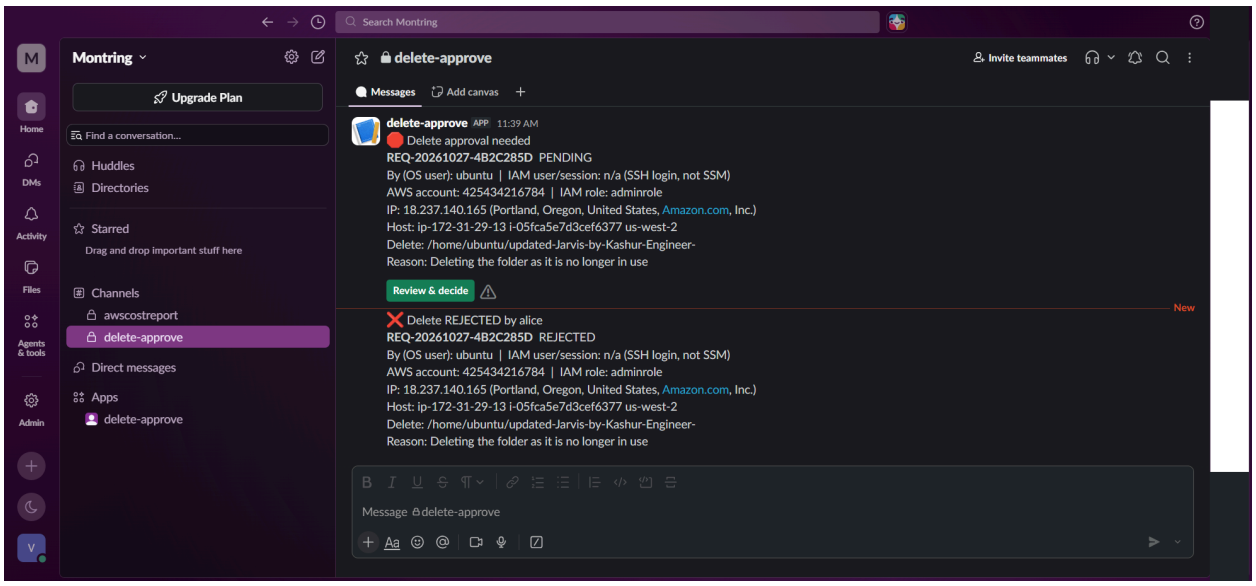
Step9: Check Slack for the Approved and Delete executed confirmation messages.



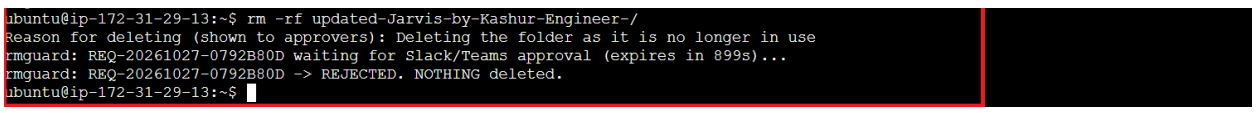
Step10: Verify the approval page shows Status: Rejected and confirms the decision was recorded.



Step11: Verify Slack shows Delete REJECTED. Message



Step12: Verify the terminal confirms REJECTED. NOTHING deleted**. Ref step1 images**



```
ubuntu@ip-172-31-29-13:~$ cat rmguard.py
#!/usr/bin/env python3
"""
rmguard.py  -  approval-gated delete for Linux / EC2   (single file, stdlib only)

WHAT IT DOES
  * `rm` on a protected path is replaced by a shim. NOTHING is deleted until a human
    approves from Slack / Teams. Reject / expiry / server down  => nothing deleted.
  * One central approval SERVER, any number of EC2 AGENTS (every instance runs the shim).
  * Every step goes to a SHA-256 hash-chained audit log (tamper-evident) with:
    who (login user, IAM instance role, SSM session owner IAM ARN), source IP, location,
    instance-id, region, private/public IP, command, reason, approver + approver IP.
  * Opening the link changes nothing; only the confirm button (POST) records a decision.
  * Requester can never approve their own request; N different approvers supported.

QUICK START (Ubuntu, python3 >= 3.8)
  Approval server (one box):
    sudo python3 rmguard.py init-server --public-url https://guard.example.com \
         --slack https://hooks.slack.com/... --teams https://...webhook...
    sudo python3 rmguard.py approver alice          # repeat for each approver
    sudo python3 rmguard.py install-server          # systemd service, append-only audit log
  Every EC2 (including the server box):
    sudo python3 rmguard.py init-agent --server-url http://<server-private-ip>:8088 --token
<TOKEN>
    sudo python3 rmguard.py install-agent           # installs /usr/local/bin/rm shim
  Test anywhere:   python3 rmguard.py selftest

OTHER COMMANDS:  verify (check audit chain) | report (CSV of audit log)
"""
import argparse, base64, csv, getpass, hashlib, hmac, html, ipaddress, json, os, pwd
import secrets, shutil, subprocess, sys, tempfile, threading, time, syslog
import urllib.error, urllib.parse, urllib.request
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

REAL_DIR = "/usr/lib/rmguard"                                  # --harden moves the real binaries
here
RM = REAL_DIR + "/rm.real" if os.path.exists(REAL_DIR + "/rm.real") else (
    "/usr/bin/rm" if os.path.exists("/usr/bin/rm") else "/bin/rm")      # the REAL rm

# ---------------------------------------------------------------------------- config
# >>> PASTE YOUR WEBHOOK URLS HERE (only needed in the copy that runs as the
approval SERVER) <<<
# Leave "" if you use `init-server --slack/--teams` or /etc/rmguard-server.json instead (those win
when set).
```

```
SLACK_WEBHOOK_URL =
"https://hooks.slack.com/services/T0B5X0AGYJZ/B0CC7HVUAUV/xSWRSej7iALFkvBYyU3oq
UA3"
TEAMS_WEBHOOK_URL = ""    # e.g. the Teams Workflows webhook URL
CONF_FILES = {"agent": ("/etc/rmguard.json", "RMGUARD_CONF"),
              "server": ("/etc/rmguard-server.json", "RMGUARD_SERVER_CONF")}
DEFAULTS = {
    "agent": {"server_url": "http://127.0.0.1:8088", "token": "",
              "protected": ["/"],                    # every file and directory on the VM
              "allow": ["/tmp", "/tmp/*", "/var/tmp", "/var/tmp/*", "/run/*", "/proc/*", "/sys/*", "/dev/*",
                        "/var/cache/*", "/var/lib/apt/*", "/var/lib/dpkg/*", "/var/lib/snapd/*",
                        "/home/*/.cache", "/home/*/.cache/*", "/root/.cache/*"]},   # never gated (glob
patterns)
    "server": {"bind": "0.0.0.0", "port": 8088, "token": "", "public_url": "http://127.0.0.1:8088",
               "data_dir": "/var/lib/rmguard", "slack_webhook": "", "teams_webhook": "",
               "approvers": {}, "required_approvals": 1, "ttl_seconds": 900, "geo_lookup": True},
}

def conf_path(kind):
    default, env = CONF_FILES[kind]
    return os.environ.get(env, default)

def load_conf(kind):
    try:
        with open(conf_path(kind)) as f:
            conf = {**DEFAULTS[kind], **json.load(f)}
    except FileNotFoundError:
        conf = dict(DEFAULTS[kind])
    if kind == "server":      # config file value wins; otherwise use the URL pasted at the top of this
file
        conf["slack_webhook"] = conf["slack_webhook"] or SLACK_WEBHOOK_URL
        conf["teams_webhook"] = conf["teams_webhook"] or TEAMS_WEBHOOK_URL
    return conf

def save_conf(kind, conf, mode):
    p = conf_path(kind)
    with open(p, "w") as f:
        json.dump(conf, f, indent=2)
    os.chmod(p, mode)

def now():
    return time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime())

def hash_pw(pw, salt=None):
```

```
    salt = salt or secrets.token_hex(8)
    return salt + "$" + hashlib.pbkdf2_hmac("sha256", pw.encode(), salt.encode(), 200000).hex()

def check_pw(stored, pw):
    if not stored or "$" not in stored:
        return False
    return hmac.compare_digest(stored, hash_pw(pw, stored.split("$")[0]))

# ---------------------------------------------------------------------------- AGENT: who/where
def imds(path, _tok=[None]):
    """EC2 metadata (IMDSv2). Returns '' when not on EC2."""
    try:
        if not _tok[0]:
            r = urllib.request.Request("http://169.254.169.254/latest/api/token", method="PUT",
                                       headers={"X-aws-ec2-metadata-token-ttl-seconds": "300"})
            _tok[0] = urllib.request.urlopen(r, timeout=1).read().decode()
        r = urllib.request.Request("http://169.254.169.254/latest/" + path,
                                   headers={"X-aws-ec2-metadata-token": _tok[0]})
        return urllib.request.urlopen(r, timeout=1).read().decode()
    except Exception:
        return ""

def login_user():
    """Original login user - survives sudo/su (kernel audit loginuid)."""
    try:
        uid = int(open("/proc/self/loginuid").read())
        if uid != 4294967295:
            return pwd.getpwuid(uid).pw_name
    except Exception:
        pass
    return os.environ.get("SUDO_USER") or getpass.getuser()

def ssh_source_ip():
    try:  # utmp record is written by sshd, so the user cannot fake it via env vars
        out = subprocess.run(["who", "am", "i"], capture_output=True, text=True, timeout=3).stdout
        if "(" in out:
            return out.rsplit("(", 1)[1].strip(") \n")
    except Exception:
        pass
    return (os.environ.get("SSH_CLIENT", "").split() or [""])[0]

def ssm_owners(instance_id, region):
    """IAM ARN(s) of active SSM Session Manager sessions on this instance (needs boto3 +
ssm:DescribeSessions)."""
```

```
    try:
        import boto3
        s = boto3.client("ssm", region_name=region).describe_sessions(
            State="Active", Filters=[{"key": "Target", "value": instance_id}])["Sessions"]
        return [x["Owner"] for x in s]
    except Exception:
        return []

def short_arn(arn):
    """arn:aws:sts::123:assumed-role/Role/alice -> assumed-role/Role/alice ;
arn:aws:iam::123:user/bob -> user/bob"""
    return arn.split(":", 5)[-1]

def context():
    az = imds("meta-data/placement/availability-zone")
    iid = imds("meta-data/instance-id")
    try:
        role = json.loads(imds("meta-data/iam/info")).get("InstanceProfileArn", "")
    except Exception:
        role = ""
    user = login_user()
    ssh_ip = ssh_source_ip()
    try:
        account = json.loads(imds("dynamic/instance-identity/document")).get("accountId", "")
    except Exception:
        account = ""
    roles = imds("meta-data/iam/security-credentials/").split()
    owners = ssm_owners(iid, az[:-1]) if iid and (user == "ssm-user" or not ssh_ip) else []
    return {"aws_account": account, "iam_role_name": roles[0] if roles else "",
            "iam_identity": [short_arn(o) for o in owners],     # IAM user / assumed-role session of
the human (SSM)
            "login_user": user, "effective_user": getpass.getuser(), "ssh_ip": ssh_ip,
            "tty": os.ttyname(0) if os.isatty(0) else "", "host": os.uname().nodename,
            "instance_id": iid, "az": az, "region": az[:-1] if az else "",
            "private_ip": imds("meta-data/local-ipv4"), "public_ip": imds("meta-data/public-ipv4"),
            "instance_role": role, "ssm_session_owners": owners,
            "cwd": os.getcwd(), "pid": os.getpid()}

def snapshot(p):
    ap = os.path.abspath(p)
    ap = os.path.join(os.path.realpath(os.path.dirname(ap)), os.path.basename(ap))  # resolve
parents, keep leaf
    try:
        st = os.lstat(ap)
```

```
    except FileNotFoundError:
        return {"path": ap, "exists": False}
    isdir = os.path.isdir(ap) and not os.path.islink(ap)
    count = 0
    if isdir:
        for _, d, f in os.walk(ap):
            count += len(d) + len(f)
            if count > 5000:
                break
    return {"path": ap, "exists": True, "type": "dir" if isdir else "file", "dev": st.st_dev,
            "inode": st.st_ino, "size": st.st_size, "mtime": int(st.st_mtime), "entries": count}

def is_protected(path, protected, allow=()):
    import fnmatch
    if any(fnmatch.fnmatch(path, a) for a in allow):
        return False
    for p in protected:
        p = p.rstrip("/")
        if path == p or path.startswith(p + "/") or p.startswith(path.rstrip("/") + "/"):
            return True
    return False

def api(conf, method, path, body=None):
    req = urllib.request.Request(conf["server_url"] + path, method=method,
                                 data=None if body is None else json.dumps(body).encode(),
                                 headers={"Content-Type": "application/json", "X-Token": conf["token"]})
    try:
        with urllib.request.urlopen(req, timeout=15) as r:
            return json.loads(r.read())
    except urllib.error.HTTPError as e:
        return {"error": e.read().decode()[:200], "http": e.code}

def err(msg):
    print("rmguard: " + msg, file=sys.stderr)

def cmd_rm(args):
    conf = load_conf("agent")
    flags, paths, end = [], [], False
    for a in args:
        if end or a == "-" or not a.startswith("-"):
            paths.append(a)
        elif a == "--":
            end = True
        else:
```

```
            flags.append(a)
    if "DPKG_MAINTSCRIPT_NAME" in os.environ:           # apt/dpkg maintainer scripts must
never hang
        os.execv(RM, [RM] + args)
    dirok = any(f in ("-d", "--dir") for f in flags)
    recursive = any(f == "--recursive" or (not f.startswith("--") and set(f) & set("rR")) for f in flags)
    free, gated = [], []
    for p in paths:
        s = snapshot(p)
        hit = s["exists"] and is_protected(s["path"], conf["protected"], conf["allow"])
        (gated if hit else free).append(s if hit else p)
    rc = 0
    if free or not paths:                               # unprotected paths: normal rm
        rc = subprocess.call([RM] + flags + ["--"] + free)
    for s in list(gated):
        if s["type"] == "dir" and not recursive and not dirok:
            err("cannot remove '%s': Is a directory (use -r)" % s["path"]); gated.remove(s); rc = 1
    if not gated:
        sys.exit(rc)

    # ---- protected: create approval request (nothing executes here)
    reason = os.environ.get("RMGUARD_REASON", "")
    if not reason and sys.stdin.isatty():
        reason = input("Reason for deleting (shown to approvers): ").strip()
    ctx = context()
    try:
        res = api(conf, "POST", "/api/requests", {"ctx": ctx, "targets": gated, "flags": flags,
                                                  "recursive": recursive, "reason": reason or "(none given)"})
        rid, expires = res["id"], res["expires"]
    except Exception as e:
        err("approval server unreachable / rejected request -> NOTHING deleted (%s)" % e);
sys.exit(1)
    syslog.syslog("rmguard %s requested by %s for %s" % (rid, ctx["login_user"], [g["path"] for g
in gated]))
    err("%s waiting for Slack/Teams approval (expires in %ds)..." % (rid, int(expires -
time.time())))

    status = "pending"
    while time.time() < expires + 20:
        try:
            status = api(conf, "GET", "/api/requests/" + rid).get("status", "pending")
        except Exception:
            status = "pending"
        if status != "pending":
```

```
            break
        time.sleep(3)
    if status != "approved":
        err("%s -> %s. NOTHING deleted." % (rid, status.upper())); sys.exit(1)

    # ---- approved: claim exactly once, revalidate, then delete
    if api(conf, "POST", "/api/requests/%s/claim" % rid).get("claimed") is not True:
        err("request already used or no longer valid. NOTHING deleted."); sys.exit(1)
    ok, detail = True, []
    for old in gated:
        new = snapshot(old["path"])
        if not new["exists"] or (new["dev"], new["inode"], new["type"]) != (old["dev"], old["inode"],
old["type"]):
            ok = False; detail.append("%s: target changed since approval, skipped" % old["path"]);
continue
        try:
            if new["type"] == "dir":
                shutil.rmtree(new["path"]) if recursive else os.rmdir(new["path"])
            else:
                os.unlink(new["path"])
            detail.append("%s: deleted" % old["path"])
        except Exception as e:
            ok = False; detail.append("%s: FAILED %s" % (old["path"], e))
    api(conf, "POST", "/api/requests/%s/result" % rid, {"ok": ok, "detail": detail})
    syslog.syslog("rmguard %s executed ok=%s %s" % (rid, ok, detail))
    err("%s approved. %s" % (rid, "; ".join(detail)))
    sys.exit(0 if ok and not rc else 1)

# ---------------------------------------------------------------------------- SERVER: state + audit
LOCK = threading.RLock()
STATE = {}

def init_state(conf):
    import sqlite3
    os.makedirs(conf["data_dir"], exist_ok=True)
    db = sqlite3.connect(os.path.join(conf["data_dir"], "requests.db"), check_same_thread=False)
    db.execute("CREATE TABLE IF NOT EXISTS req (id TEXT PRIMARY KEY, data TEXT)")
    STATE["db"] = db
    STATE["audit"] = os.path.join(conf["data_dir"], "audit.jsonl")
    last = "0" * 64
    try:
        for line in open(STATE["audit"]):
            last = json.loads(line)["hash"]
    except FileNotFoundError:
```

```
        pass
    STATE["last"] = last

def audit(event, **kw):
    """Append one hash-chained record. Editing/deleting any past line breaks the chain (see
`verify`)."""
    with LOCK:
        rec = {"ts": now(), "event": event, **kw, "prev": STATE["last"]}
        rec["hash"] = hashlib.sha256(json.dumps(rec, sort_keys=True).encode()).hexdigest()
        with open(STATE["audit"], "a") as f:
            f.write(json.dumps(rec, sort_keys=True) + "\n"); f.flush(); os.fsync(f.fileno())
        STATE["last"] = rec["hash"]

def save(d, new=False):
    with LOCK:
        STATE["db"].execute("INSERT OR REPLACE INTO req VALUES (?,?)", (d["id"],
json.dumps(d)))
        STATE["db"].commit()

def load(rid):
    with LOCK:
        row = STATE["db"].execute("SELECT data FROM req WHERE id=?", (rid,)).fetchone()
        if not row:
            return None
        d = json.loads(row[0])
        if d["status"] == "pending" and time.time() > d["expires"]:
            d["status"] = "expired"; save(d); audit("expired", id=d["id"])
        return d

def geo(ip):
    try:
        if not ip or ipaddress.ip_address(ip).is_private:
            return "private network / VPN"
        j = json.loads(urllib.request.urlopen(
            "http://ip-api.com/json/%s?fields=country,regionName,city,isp" % ip, timeout=3).read())
        return ", ".join(x for x in (j.get("city"), j.get("regionName"), j.get("country"), j.get("isp")) if x)
    except Exception:
        return "unknown"

def post_json(url, payload):
    if not url:
        return False
    try:
        r = urllib.request.urlopen(urllib.request.Request(url, data=json.dumps(payload).encode(),
```

```
                                headers={"Content-Type": "application/json"}), timeout=10)
        return 200 <= r.status < 300
    except Exception as e:
        print("notify failed:", e, file=sys.stderr)
        return False

def summary(d):
    c = d["ctx"]
    iam = ", ".join(c.get("iam_identity") or []) or "n/a (SSH login, not SSM)"
    return ("*%s*  %s\nBy (OS user): %s  |  IAM user/session: %s\nAWS account: %s  |  IAM role:
%s\nIP: %s (%s)\nHost: %s %s %s\nDelete: %s\nReason: %s" % (
        d["id"], d["status"].upper(), c.get("login_user", "?"), iam,
        c.get("aws_account") or "n/a", c.get("iam_role_name") or "n/a",
        c.get("ssh_ip") or "n/a", d.get("location", ""),
        c.get("host"), c.get("instance_id"), c.get("region"),
        ", ".join(t["path"] for t in d["targets"]), d["reason"]))

def notify(conf, d, headline, with_link=True):
    text = "%s\n%s" % (headline, summary(d))
    link = "%s/r/%s" % (conf["public_url"].rstrip("/"), d["id"])
    slack = {"text": text}
    teams_body = [{"type": "TextBlock", "text": headline, "weight": "Bolder", "wrap": True},
                  {"type": "TextBlock", "text": summary(d).replace("*", ""), "wrap": True}]
    teams_act = []
    if with_link:
        slack["blocks"] = [{"type": "section", "text": {"type": "mrkdwn", "text": text}},
                           {"type": "actions", "elements": [{"type": "button", "style": "primary",
                            "text": {"type": "plain_text", "text": "Review & decide"}, "url": link}]}]
        teams_act = [{"type": "Action.OpenUrl", "title": "Review & decide", "url": link}]
    teams = {"type": "message", "attachments": [{"contentType":
"application/vnd.microsoft.card.adaptive",
             "content": {"$schema": "http://adaptivecards.io/schemas/adaptive-card.json", "type":
"AdaptiveCard",
                         "version": "1.4", "body": teams_body, "actions": teams_act}}]}
    return {"slack": post_json(conf["slack_webhook"], slack), "teams":
post_json(conf["teams_webhook"], teams)}

class H(BaseHTTPRequestHandler):
    def log_message(self, *a):
        pass

    def out(self, code, body, ctype="application/json", extra=None):
        b = body.encode() if isinstance(body, str) else json.dumps(body).encode()
        self.send_response(code); self.send_header("Content-Type", ctype)
```

```
        self.send_header("Content-Length", str(len(b)))
        for k, v in (extra or {}).items():
            self.send_header(k, v)
        self.end_headers(); self.wfile.write(b)

    def body(self):
        n = min(int(self.headers.get("Content-Length") or 0), 1_000_000)
        return self.rfile.read(n).decode() if n else ""

    def agent_ok(self, conf):
        return conf["token"] and hmac.compare_digest(self.headers.get("X-Token", ""),
conf["token"])

    def approver(self, conf):
        h = self.headers.get("Authorization", "")
        if h.startswith("Basic "):
            try:
                u, _, p = base64.b64decode(h[6:]).decode().partition(":")
                if check_pw(conf["approvers"].get(u), p):
                    return u
            except Exception:
                pass
        self.out(401, "login required", "text/plain", {"WWW-Authenticate": 'Basic realm="rmguard
approvers"'})
        return None

    # ---- GET
    def do_GET(self):
        conf = load_conf("server"); parts = self.path.split("?")[0].strip("/").split("/")
        if parts[0] == "api" and len(parts) == 3:
            if not self.agent_ok(conf):
                return self.out(403, {"error": "bad token"})
            d = load(parts[2])
            return self.out(200, {"status": d["status"]}) if d else self.out(404, {"error": "unknown"})
        if parts[0] == "r" and len(parts) == 2:
            u = self.approver(conf)
            if u:
                d = load(parts[1])
                return self.page(d, u) if d else self.out(404, "unknown request", "text/plain")
            return
        if parts[0] == "audit":
            if self.approver(conf):
                with open(STATE["audit"]) as f:
                    return self.out(200, "".join(f.readlines()[-200:]), "text/plain")
```

```
            return
        self.out(200, "rmguard ok", "text/plain")

    def page(self, d, user, msg=""):
        c, e = d["ctx"], html.escape
        rows = [("Request", d["id"]), ("Status", d["status"]), ("Requested by (login user)",
d["requester"]),
                ("Reason", d["reason"]), ("Delete", "\n".join(t["path"] + " (%s, %s entries)" % (t["type"],
t["entries"]) for t in d["targets"])),
                ("Source IP / location", "%s  /  %s" % (c.get("ssh_ip") or "n/a", d.get("location"))),
                ("Instance", "%s  %s  %s" % (c.get("host"), c.get("instance_id"), c.get("region"))),
                ("Instance IPs", "private %s  public %s" % (c.get("private_ip"), c.get("public_ip"))),
                ("AWS account", c.get("aws_account")), ("IAM role name", c.get("iam_role_name")),
("IAM instance profile", c.get("instance_role")),
                ("IAM user / session (SSM)", '', ''.join(c.get("ssm_session_owners") or []) or "n/a (SSH
login)"),
                ("Approvals", "%d of %d: %s" % (len(d["approvals"]), d["required"], ', '.join(a["by"] for a
in d["approvals"])))]
        tbl = "".join("<tr><th align=left>%s</th><td><pre>%s</pre></td></tr>" % (e(k), e(str(v))) for
k, v in rows)
        form = ""
        if d["status"] == "pending":
            form = ('<form method=post><button name=decision value=approve
style="background:#1a7f37;color:#fff;padding:10px 24px">Approve</button> '
                    '<button name=decision value=reject
style="background:#cf222e;color:#fff;padding:10px 24px">Reject</button></form>')
        self.out(200, "<html><body style='font-family:sans-serif'><h2>Delete
approval</h2><p>Signed in as
<b>%s</b></p><p>%s</p><table>%s</table>%s</body></html>"
                 % (e(user), e(msg), tbl, form), "text/html")

    # ---- POST
    def do_POST(self):
        conf = load_conf("server"); parts = self.path.split("?")[0].strip("/").split("/")
        if parts[0] == "r" and len(parts) == 2:
            return self.decide(conf, parts[1])
        if not (parts[0] == "api" and self.agent_ok(conf)):
            return self.out(403, {"error": "bad token"})
        raw = self.body()
        body = json.loads(raw) if raw else {}
        if parts[1] == "requests" and len(parts) == 2:
            return self.create(conf, body)
        d = load(parts[2]) if len(parts) == 4 else None
        if not d:
```

```
            return self.out(404, {"error": "unknown"})
        with LOCK:
            d = load(parts[2])
            if parts[3] == "claim":                          # atomic: approved -> executing, only once
                if d["status"] != "approved":
                    return self.out(200, {"claimed": False})
                d["status"] = "executing"; save(d); audit("claimed", id=d["id"])
                return self.out(200, {"claimed": True})
            if parts[3] == "result":
                d["status"] = "executed" if body.get("ok") else "failed"; save(d)
                audit("executed" if body.get("ok") else "failed", id=d["id"], detail=body.get("detail"),
                      ctx=d["ctx"], location=d.get("location"))
                notify(conf, d, "%s Delete %s" % ("✅" if body.get("ok") else "⚠️", d["status"]),
with_link=False)
                return self.out(200, {"ok": True})
        self.out(404, {"error": "unknown"})

    def create(self, conf, body):
        ctx = body["ctx"]
        d = {"id": "REQ-%s-%s" % (time.strftime("%Y%m%d"), secrets.token_hex(4).upper()),
"status": "pending",
             "created": now(), "expires": time.time() + conf["ttl_seconds"], "requester":
ctx.get("login_user"),
             "reason": body.get("reason", ""), "targets": body["targets"], "flags": body.get("flags", []),
             "ctx": ctx, "agent_ip": self.client_address[0], "approvals": [], "required":
conf["required_approvals"],
             "location": geo(ctx.get("ssh_ip")) if conf["geo_lookup"] else ""}
        save(d)
        audit("request_created", id=d["id"], requester=d["requester"], reason=d["reason"], ctx=ctx,
              targets=[t["path"] for t in d["targets"]], agent_ip=d["agent_ip"], location=d["location"])
        notify(conf, d, "🛑 Delete approval needed")
        self.out(200, {"id": d["id"], "expires": d["expires"]})

    def decide(self, conf, rid):
        user = self.approver(conf)
        if not user:
            return
        form = urllib.parse.parse_qs(self.body())
        decision = (form.get("decision") or [""])[0]
        src = self.headers.get("X-Forwarded-For") or self.client_address[0]
        with LOCK:
            d = load(rid)
            if not d:
                return self.out(404, "unknown", "text/plain")
```

```
            if d["status"] != "pending":
                return self.page(d, user, "Request is already %s." % d["status"])
            if user == d["requester"]:
                audit("denied_self_approval", id=rid, approver=user, approver_ip=src)
                return self.page(d, user, "You cannot approve your own request.")
            if any(a["by"] == user for a in d["approvals"]):
                return self.page(d, user, "You already approved this.")
            if decision == "reject":
                d["status"] = "rejected"; audit("rejected", id=rid, approver=user, approver_ip=src)
                save(d); notify(conf, d, "❌ Delete REJECTED by %s" % user, with_link=False)
            elif decision == "approve":
                d["approvals"].append({"by": user, "ip": src, "ts": now()})
                audit("approved", id=rid, approver=user, approver_ip=src, count=len(d["approvals"]),
need=d["required"])
                if len(d["approvals"]) >= d["required"]:
                    d["status"] = "approved"
                save(d); notify(conf, d, "👍 Approved by %s (%d/%d)" % (user, len(d["approvals"]),
d["required"]),
                                with_link=d["status"] == "pending")
        self.page(d, user, "Decision recorded.")

def start_server():
    conf = load_conf("server")
    init_state(conf)
    httpd = ThreadingHTTPServer((conf["bind"], conf["port"]), H)
    return httpd

# ---------------------------------------------------------------------- admin commands
def cmd_init_server(a):
    c = load_conf("server")
    c["token"] = c["token"] or secrets.token_urlsafe(32)
    for k, v in (("public_url", a.public_url), ("slack_webhook", a.slack), ("teams_webhook",
a.teams)):
        if v:
            c[k] = v
    save_conf("server", c, 0o600)
    print("server config:", conf_path("server"), "\nAGENT TOKEN (use in init-agent):", c["token"])

def cmd_init_agent(a):
    c = load_conf("agent"); c["server_url"] = a.server_url; c["token"] = a.token
    save_conf("agent", c, 0o644)     # readable so non-root users' rm can reach the server; token
cannot approve anything
    print("agent config:", conf_path("agent"))
```

```
def cmd_approver(a):
    c = load_conf("server"); pw = getpass.getpass("password for %s: " % a.name)
    c["approvers"][a.name] = hash_pw(pw); save_conf("server", c, 0o600); print("approver
saved:", a.name)

SHIMS = (("rm", ""), ("rmdir", "-d"), ("unlink", ""))   # name, extra flag passed to the guarded rm

def shim_text(flag):
    return '#!/bin/sh\nexec /usr/bin/python3 /opt/rmguard/rmguard.py rm %s "$@"\n' % flag

def divert(target, real, text):
    """Move the real binary aside (dpkg-divert survives package upgrades) and put the shim at its
path."""
    os.makedirs(os.path.dirname(real), exist_ok=True)
    if not os.path.exists(real):
        subprocess.check_call(["dpkg-divert", "--local", "--rename", "--divert", real, "--add", target])
    with open(target, "w") as f:
        f.write(text)
    os.chmod(target, 0o755)

def undivert(target, real):
    if os.path.exists(real):
        if os.path.exists(target):
            os.unlink(target)
        subprocess.check_call(["dpkg-divert", "--local", "--rename", "--remove", target])

def cmd_install_agent(a):
    os.makedirs("/opt/rmguard", exist_ok=True)
    shutil.copy(os.path.abspath(__file__), "/opt/rmguard/rmguard.py");
os.chmod("/opt/rmguard/rmguard.py", 0o755)
    for name, flag in SHIMS:
        if a.harden:   # replaces /usr/bin/rm itself: /usr/bin/rm, /bin/rm and absolute paths are ALL
gated
            divert("/usr/bin/" + name, "%s/%s.real" % (REAL_DIR, name), shim_text(flag))
        else:          # PATH shim only: /usr/local/bin comes first for users and for sudo on Ubuntu
            with open("/usr/local/bin/" + name, "w") as f:
                f.write(shim_text(flag))
            os.chmod("/usr/local/bin/" + name, 0o755)
    print("installed rm / rmdir / unlink guards (%s). Test: touch ~/x && rm ~/x" % ("HARDENED" if
a.harden else "PATH shim"))

def cmd_uninstall_agent(_):
    for name, _f in SHIMS:
        if os.path.exists("/usr/local/bin/" + name):
```

```
            os.unlink("/usr/local/bin/" + name)
        undivert("/usr/bin/" + name, "%s/%s.real" % (REAL_DIR, name))
    print("rmguard removed, original rm restored")

def cmd_install_server(_):
    os.makedirs("/opt/rmguard", exist_ok=True)
    shutil.copy(os.path.abspath(__file__), "/opt/rmguard/rmguard.py")
    open("/etc/systemd/system/rmguard.service", "w").write(
        "[Unit]\nDescription=rmguard approval server\nAfter=network.target\n[Service]\n"
        "ExecStart=/usr/bin/python3 /opt/rmguard/rmguard.py
serve\nRestart=always\n[Install]\nWantedBy=multi-user.target\n")
    c = load_conf("server"); os.makedirs(c["data_dir"], exist_ok=True)
    audit_file = os.path.join(c["data_dir"], "audit.jsonl"); open(audit_file, "a").close()
    subprocess.call(["chattr", "+a", audit_file])     # append-only: even root cannot truncate it
without chattr -a
    subprocess.call(["systemctl", "daemon-reload"]); subprocess.call(["systemctl", "enable",
"--now", "rmguard"])
    print("server running on port", c["port"])

def cmd_notify_test(_):
    """Send a sample approval card to Slack + Teams so you can see exactly what approvers
get."""
    conf = load_conf("server")
    d = {"id": "REQ-TEST-0001", "status": "pending", "reason": "notify-test", "targets": [{"path":
"/etc/demo-file"}],
         "ctx": {"login_user": login_user(), "ssh_ip": ssh_source_ip(), "host": os.uname().nodename,
                 "instance_id": imds("meta-data/instance-id"), "region": ""}, "location": "test"}
    print("slack webhook set:", bool(conf["slack_webhook"]), "| teams webhook set:",
bool(conf["teams_webhook"]))
    print("result:", notify(conf, d, "🛑 Delete approval needed (TEST)"))

def read_audit():
    for line in open(os.path.join(load_conf("server")["data_dir"], "audit.jsonl")):
        yield json.loads(line)

def cmd_verify(_):
    last, n = "0" * 64, 0
    for rec in read_audit():
        h = rec.pop("hash")
        if rec["prev"] != last or hashlib.sha256(json.dumps(rec,
sort_keys=True).encode()).hexdigest() != h:
            print("TAMPERED at record", n + 1, rec.get("ts")); sys.exit(2)
        last, n = h, n + 1
    print("audit log OK: %d records, head hash %s" % (n, last))
```

```
def cmd_report(_):
    w = csv.writer(sys.stdout)
    w.writerow(["time", "event", "request", "user/approver", "source_ip", "location", "instance",
"region", "iam_role", "ssm_owner", "aws_account", "iam_role_name", "iam_user_session",
"targets/detail"])
    for r in read_audit():
        c = r.get("ctx") or {}
        w.writerow([r["ts"], r["event"], r.get("id"), r.get("requester") or r.get("approver"),
c.get("ssh_ip") or r.get("approver_ip"),
                    r.get("location"), c.get("instance_id"), c.get("region"), c.get("instance_role"),
                    ";".join(c.get("ssm_session_owners") or []), c.get("aws_account"),
c.get("iam_role_name"),
                    ";".join(c.get("iam_identity") or []), json.dumps(r.get("targets") or r.get("detail") or "")])

def cmd_selftest(_):
    tmp = tempfile.mkdtemp(prefix="rmguard-test-"); port = 18088; pw = "test-pass"
    os.makedirs(tmp + "/protected")
    json.dump({"port": port, "bind": "127.0.0.1", "token": "t0k", "data_dir": tmp + "/data",
"geo_lookup": False,
               "approvers": {"alice": hash_pw(pw)}, "ttl_seconds": 60, "public_url":
"http://127.0.0.1:%d" % port},
              open(tmp + "/server.json", "w"))
    json.dump({"server_url": "http://127.0.0.1:%d" % port, "token": "t0k", "protected": [tmp +
"/protected"], "allow": []},
              open(tmp + "/agent.json", "w"))
    os.environ["RMGUARD_SERVER_CONF"], os.environ["RMGUARD_CONF"] = tmp +
"/server.json", tmp + "/agent.json"
    os.environ["RMGUARD_REASON"] = "selftest"
    httpd = start_server(); threading.Thread(target=httpd.serve_forever, daemon=True).start()
    auth = "Basic " + base64.b64encode(("alice:" + pw).encode()).decode()

    def attempt(decision):
        f = "%s/protected/%s.txt" % (tmp, decision); open(f, "w").write("x")
        p = subprocess.Popen([sys.executable, os.path.abspath(__file__), "rm", f],
stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True)
        rid = None
        for _ in range(60):
            for (i,) in STATE["db"].execute("SELECT id FROM req").fetchall():
                d = load(i)
                if d["status"] == "pending" and d["targets"][0]["path"] == os.path.realpath(f):
                    rid = i
            if rid:
                break
```

```
            time.sleep(0.25)
        assert rid, "request never created"
        # GET must NOT change anything
        urllib.request.urlopen(urllib.request.Request("http://127.0.0.1:%d/r/%s" % (port, rid),
headers={"Authorization": auth}))
        assert load(rid)["status"] == "pending", "GET changed state!"
        urllib.request.urlopen(urllib.request.Request("http://127.0.0.1:%d/r/%s" % (port, rid),
method="POST",
                              data=urllib.parse.urlencode({"decision": decision}).encode(),
headers={"Authorization": auth}))
        p.wait(timeout=30)
        return os.path.exists(f), p.returncode

    def check(name, cond):
        print(("PASS " if cond else "FAIL ") + name); return cond
    ok = True
    exists, rc = attempt("approve"); ok &= check("approved  -> file deleted, exit 0", not exists and
rc == 0)
    exists, rc = attempt("reject");  ok &= check("rejected  -> file kept,    exit 1", exists and rc == 1)
    free = tmp + "/free.txt"; open(free, "w").write("x")
    subprocess.call([sys.executable, os.path.abspath(__file__), "rm", free])
    ok &= check("unprotected path -> deleted immediately", not os.path.exists(free))
    last, n = "0" * 64, 0
    for r in read_audit():
        h = r.pop("hash"); ok &= (r["prev"] == last and hashlib.sha256(json.dumps(r,
sort_keys=True).encode()).hexdigest() == h); last = h; n += 1
    check("audit hash chain valid (%d records)" % n, ok)
    httpd.shutdown(); shutil.rmtree(tmp, ignore_errors=True)
    print("ALL TESTS PASSED" if ok else "SOME TESTS FAILED"); sys.exit(0 if ok else 1)

def main():
    if len(sys.argv) > 1 and sys.argv[1] == "rm":      # shim path: keep rm's own args untouched
        return cmd_rm(sys.argv[2:])
    ap = argparse.ArgumentParser(description=__doc__,
formatter_class=argparse.RawDescriptionHelpFormatter)
    sp = ap.add_subparsers(dest="cmd", required=True)
    s = sp.add_parser("init-server"); s.add_argument("--public-url"); s.add_argument("--slack");
s.add_argument("--teams"); s.set_defaults(f=cmd_init_server)
    s = sp.add_parser("init-agent"); s.add_argument("--server-url", required=True);
s.add_argument("--token", required=True); s.set_defaults(f=cmd_init_agent)
    s = sp.add_parser("approver"); s.add_argument("name"); s.set_defaults(f=cmd_approver)
    s = sp.add_parser("install-agent"); s.add_argument("--harden", action="store_true",
help="also gate /usr/bin/rm and absolute paths"); s.set_defaults(f=cmd_install_agent)
```

```
    for name, fn in (("uninstall-agent", cmd_uninstall_agent), ("install-server", cmd_install_server),
("verify", cmd_verify),
                     ("report", cmd_report), ("selftest", cmd_selftest), ("notify-test", cmd_notify_test)):
        sp.add_parser(name).set_defaults(f=fn)
    sp.add_parser("serve").set_defaults(f=lambda _: (lambda h: (print("rmguard server on",
h.server_address), h.serve_forever()))(start_server()))
    a = ap.parse_args(); a.f(a)

if __name__ == "__main__":
    main()
```

Part 0: Prepare (once, in the AWS console)

1. Pick one EC2 as the **approval server**. Note its **private IP** (`SERVER_PRIVATE_IP`).
2. In its security group, add two inbound rules:
    - Custom TCP, port `8088`, source **My IP**,to openg the approval page in your browser.
    - Custom TCP, port `8088`, source **your VPC range or the other instances' security group**, so the agents can reach it.
3. Get a Slack incoming webhook URL (api.slack.com/apps, then Incoming Webhooks, then Add New Webhook to Workspace).
4. Put the file on each EC2:

scp -i key.pem rmguard.py ubuntu@EC2_PUBLIC_IP:~/

Part 1: Approval server (on the server EC2 only)

1. Test the file

python3 ~/rmguard.py selftest

Expected: `ALL TESTS PASSED`.

2. Create the server config

sudo python3 ~/rmguard.py init-server --public-url auto --slack "YOUR_SLACK_WEBHOOK_URL"

Expected: it prints an `AGENT TOKEN`. Copy it, because every agent needs it. (If the Slack URL is already pasted inside the file, you can leave out `--slack`.)

3. Create an approver login (the name must not be `ubuntu`)

sudo python3 ~/rmguard.py approver alice

Type a password. Expected: `approver saved: alice`.

4. Start the server

sudo python3 ~/rmguard.py install-server
sudo systemctl status rmguard --no-pager

Expected: `active (running)`.

5. Test Slack

sudo python3 /opt/rmguard/rmguard.py notify-test

Expected: `slack: True` and a card in your channel. Click its button. A sign-in box should open.

Part 2: Agent (on every EC2 you want guarded, including the server)

1. Test the file

python3 ~/rmguard.py selftest

2. Check it can reach the server

curl http://SERVER_PRIVATE_IP:8088/

Expected: `rmguard ok`. If it hangs, fix the security group rule from Part 0.

3. Turn on the guard

sudo python3 ~/rmguard.py init-agent --server-url http://SERVER_PRIVATE_IP:8088 --token YOUR_TOKEN
sudo python3 ~/rmguard.py install-agent

Expected: `installed rm / rmdir / unlink guards (PATH shim)`.

4. Test an approve

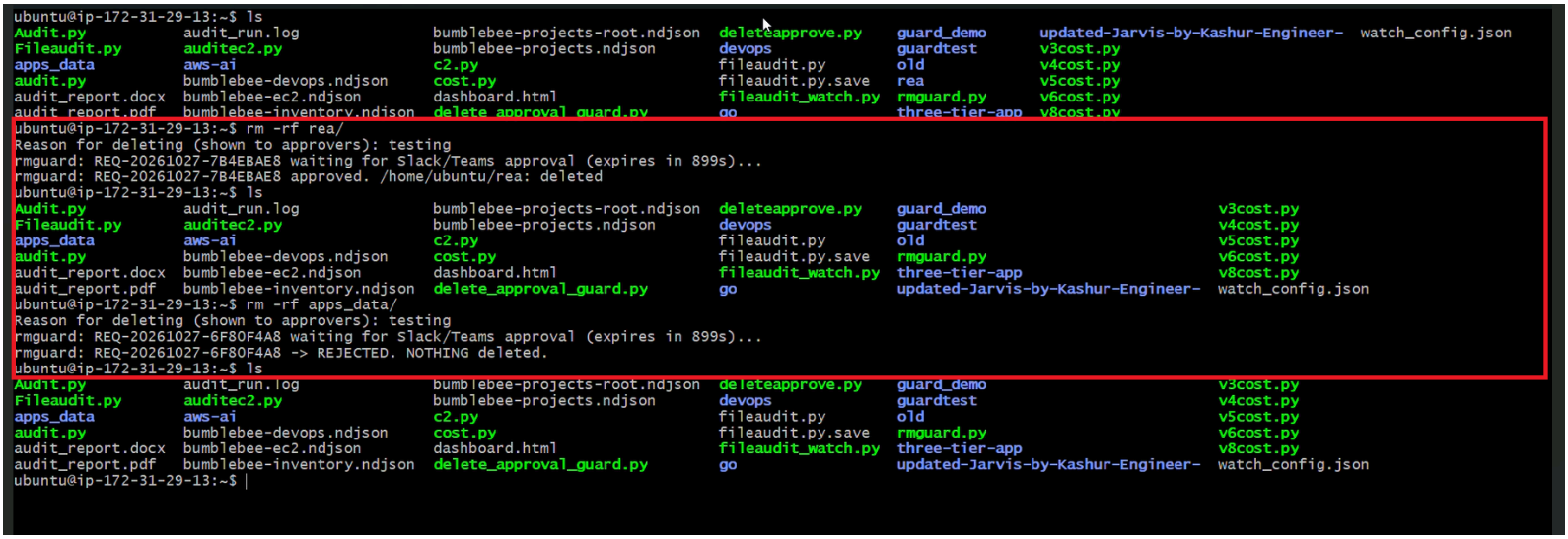
touch ~/t.txt
rm ~/t.txt

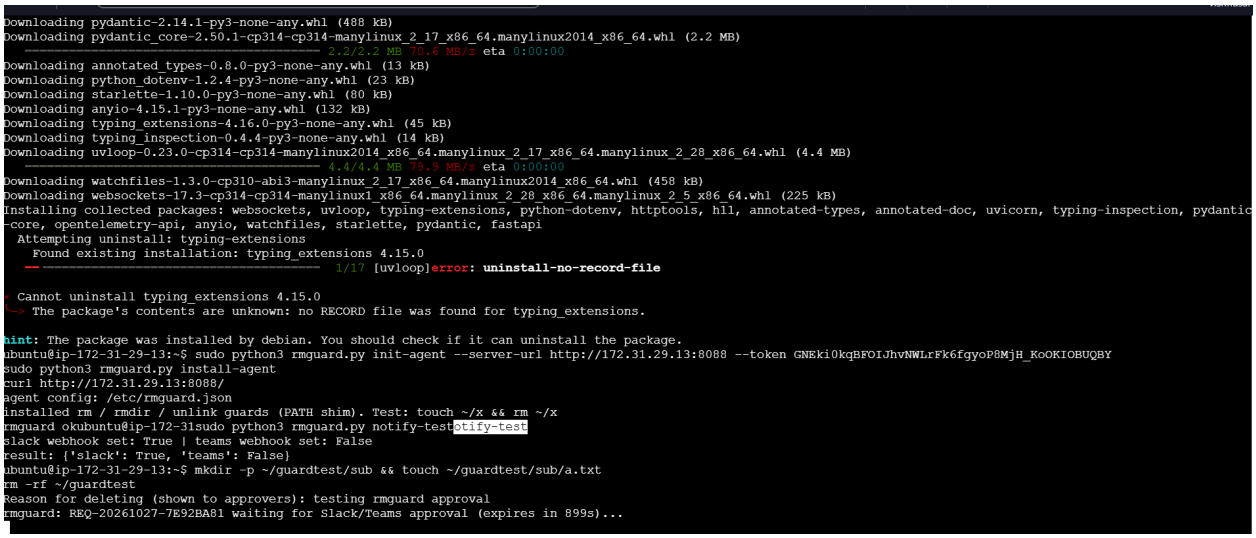
Type a reason. In the Slack card, click Review & decide, log in as `alice`, and click Approve. Then `ls ~/t.txt` should say it doesn't exist.

5. Test a reject

touch ~/t2.txt
rm ~/t2.txt

Click Reject. Expected: `REJECTED. NOTHING deleted.` and the file is still there.


ubuntu@ip-172-31-29-13:~$ ls
Audit.py audit_run.log bumblebee-projects-root.ndjson deleteapprove.py guard_demo updated-Jarvis-by-Kashur-Engineer- watch_config.json
Fileaudit.py auditec2.py bumblebee-projects.ndjson devops guardtest v3cost.py
apps_data aws-ai c2.py fileaudit.py old v4cost.py
audit.py bumblebee-devops.ndjson cost.py fileaudit.py.save rea v5cost.py
audit_report.docx bumblebee-ec2.ndjson dashboard.html fileaudit_watch.py rmguard.py v6cost.py
audit_report.pdf bumblebee-inventory.ndjson delete_approval_guard.py go three-tier-app v8cost.py
ubuntu@ip-172-31-29-13:~$ rm -rf rea/
Reason for deleting (shown to approvers): testing
rmguard: REQ-20261027-7B4EBAE8 waiting for Slack/Teams approval (expires in 899s)...
rmguard: REQ-20261027-7B4EBAE8 approved. /home/ubuntu/rea: deleted
ubuntu@ip-172-31-29-13:~$ ls
Audit.py audit_run.log bumblebee-projects-root.ndjson deleteapprove.py guard_demo v3cost.py
Fileaudit.py auditec2.py bumblebee-projects.ndjson devops guardtest v4cost.py
apps_data aws-ai c2.py fileaudit.py old v5cost.py
audit.py bumblebee-devops.ndjson cost.py fileaudit.py.save rmguard.py v6cost.py
audit_report.docx bumblebee-ec2.ndjson dashboard.html fileaudit_watch.py three-tier-app v8cost.py
audit_report.pdf bumblebee-inventory.ndjson delete_approval_guard.py go updated-Jarvis-by-Kashur-Engineer- watch_config.json
ubuntu@ip-172-31-29-13:~$ rm -rf apps_data/
Reason for deleting (shown to approvers): testing
rmguard: REQ-20261027-6F80F4A8 waiting for Slack/Teams approval (expires in 899s)...
rmguard: REQ-20261027-6F80F4A8 -> REJECTED. NOTHING deleted.
ubuntu@ip-172-31-29-13:~$ ls
Audit.py audit_run.log bumblebee-projects-root.ndjson deleteapprove.py guard_demo v3cost.py
Fileaudit.py auditec2.py bumblebee-projects.ndjson devops guardtest v4cost.py
apps_data aws-ai c2.py fileaudit.py old v5cost.py
audit.py bumblebee-devops.ndjson cost.py fileaudit.py.save rmguard.py v6cost.py
audit_report.docx bumblebee-ec2.ndjson dashboard.html fileaudit_watch.py three-tier-app v8cost.py
audit_report.pdf bumblebee-inventory.ndjson delete_approval_guard.py go updated-Jarvis-by-Kashur-Engineer- watch_config.json
ubuntu@ip-172-31-29-13:~$


Downloading pydantic-2.14.1-py3-none-any.whl (488 kB)
Downloading pydantic_core-2.50.1-cp314-cp314-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.2 MB)
2.2/2.2 MB 70.6 MB/s eta 0:00:00
Downloading annotated_types-0.8.0-py3-none-any.whl (13 kB)
Downloading python_dotenv-1.2.4-py3-none-any.whl (23 kB)
Downloading starlette-1.10.0-py3-none-any.whl (80 kB)
Downloading anyio-4.15.1-py3-none-any.whl (132 kB)
Downloading typing_extensions-4.16.0-py3-none-any.whl (45 kB)
Downloading typing_inspection-0.4.4-py3-none-any.whl (14 kB)
Downloading uvloop-0.23.0-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (4.4 MB)
4.4/4.4 MB 78.8 MB/s eta 0:00:00
Downloading watchfiles-1.3.0-cp310-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (458 kB)
Downloading websockets-17.3-cp314-cp314-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (225 kB)
Installing collected packages: websockets, uvloop, typing-extensions, python-dotenv, httptools, h11, annotated-types, annotated-doc, uvicorn, typing-inspection, pydantic
-core, opentelemetry-api, anyio, watchfiles, starlette, pydantic, fastapi
Attempting uninstall: typing-extensions
Found existing installation: typing_extensions 4.15.0
1/17 [uvloop]error: uninstall-no-record-file
Cannot uninstall typing_extensions 4.15.0
The package's contents are unknown: no RECORD file was found for typing_extensions.
hint: The package was installed by debian. You should check if it can uninstall the package.
ubuntu@ip-172-31-29-13:~$ sudo python3 rmguard.py init-agent --server-url http://172.31.29.13:8088 --token GNEki0kqBFOIJhvNWLrFk6fgyoP8MjH_KoOKIOBUQBY
sudo python3 rmguard.py install-agent
curl http://172.31.29.13:8088/
agent config: /etc/rmguard.json
installed rm / rmdir / unlink guards (PATH shim). Test: touch ~/x && rm ~/x
rmguard okubuntu@ip-172-31sudo python3 rmguard.py notify-testotify-test
slack webhook set: True | teams webhook set: False
result: {'slack': True, 'teams': False}
ubuntu@ip-172-31-29-13:~$ mkdir -p ~/guardtest/sub && touch ~/guardtest/sub/a.txt
rm -rf ~/guardtest
Reason for deleting (shown to approvers): testing rmguard approval
rmguard: REQ-20261027-7E92BA81 waiting for Slack/Teams approval (expires in 899s)...

```
python3 rmguard.py selftest          # test first, no setup needed

# server box
sudo python3 rmguard.py init-server --public-url http://<server-ip>:8088 --slack
<slack-webhook> --teams <teams-webhook>
sudo python3 rmguard.py approver alice
sudo python3 rmguard.py install-server

# every EC2 (token is printed by init-server)
sudo python3 rmguard.py init-agent --server-url http://<server-private-ip>:8088 --token
<TOKEN>
sudo python3 rmguard.py install-agent
sudo mkdir /etc/rmguard-test && sudo rm -rf /etc/rmguard-test    # real test

python3 rmguard.py verify          # detects any tampering of the audit log
python3 rmguard.py report > audit.csv
```

Token

You create the token on the approval server (`mlop`). It is made for you, so there is nothing to invent.

Step 1: Log in to `mlop`
Open your SSH session to the approval server.

Step 2: Run the create command

```
sudo python3 ~/rmguard.py init-server --public-url auto
```

The program makes a random token the first time and saves it in **`/etc/rmguard-server.json`**.

Step 3: Copy the token
The last line of the output looks like this:

```
AGENT TOKEN (use in init-agent): GNEki0kq...
```

Copy everything after the colon. Keep it private, because it is a password for your servers.

Step 4: If you missed it, show it again

```
sudo grep '"token"' /etc/rmguard-server.json
```

The long text inside the quotes is your token.

Step 5: Use it on each server that should be guarded

Run this on every guarded server, including `mlop`, and put your token where it says `PASTE_TOKEN_HERE`:

```
sudo python3 ~/rmguard.py init-agent --server-url http://172.31.29.13:8088 --token PASTE_TOKEN_HERE
```

To make a brand new token

Run these on `mlop`:

```
sudo python3 - <<'EOF'
import json
p = "/etc/rmguard-server.json"
c = json.load(open(p)); c["token"] = ""
json.dump(c, open(p, "w"), indent=2)
EOF
sudo python3 ~/rmguard.py init-server
```

It prints a new token, and the old one stops working at once. Then run Step 5 again with the new token on every guarded server, or their `rm` will fail until you do.

Check that the two tokens match

On `mlop`:

```
sudo grep '"token"' /etc/rmguard-server.json
```

On the other server:

```
sudo grep '"token"' /etc/rmguard.json
```

Both lines must show the same value. Then `curl http://172.31.29.13:8088/` should print `rmguard ok`.

RMGuard Agent — Simple Explanation

RMGuard Agent is installed on each EC2 instance to intercept deletion commands and request approval before deleting protected files.

How it works

1. User on EC2

Runs `rm -rf /app/logs`

2. RMGuard Agent

Checks whether the path requires approval and sends a request to the central server.

3. Slack / Teams

Approver selects Approve or Reject.

Approve

Agent attempts deletion.

Reject

Guarded deletion is cancelled.

What does the agent do on each EC2?

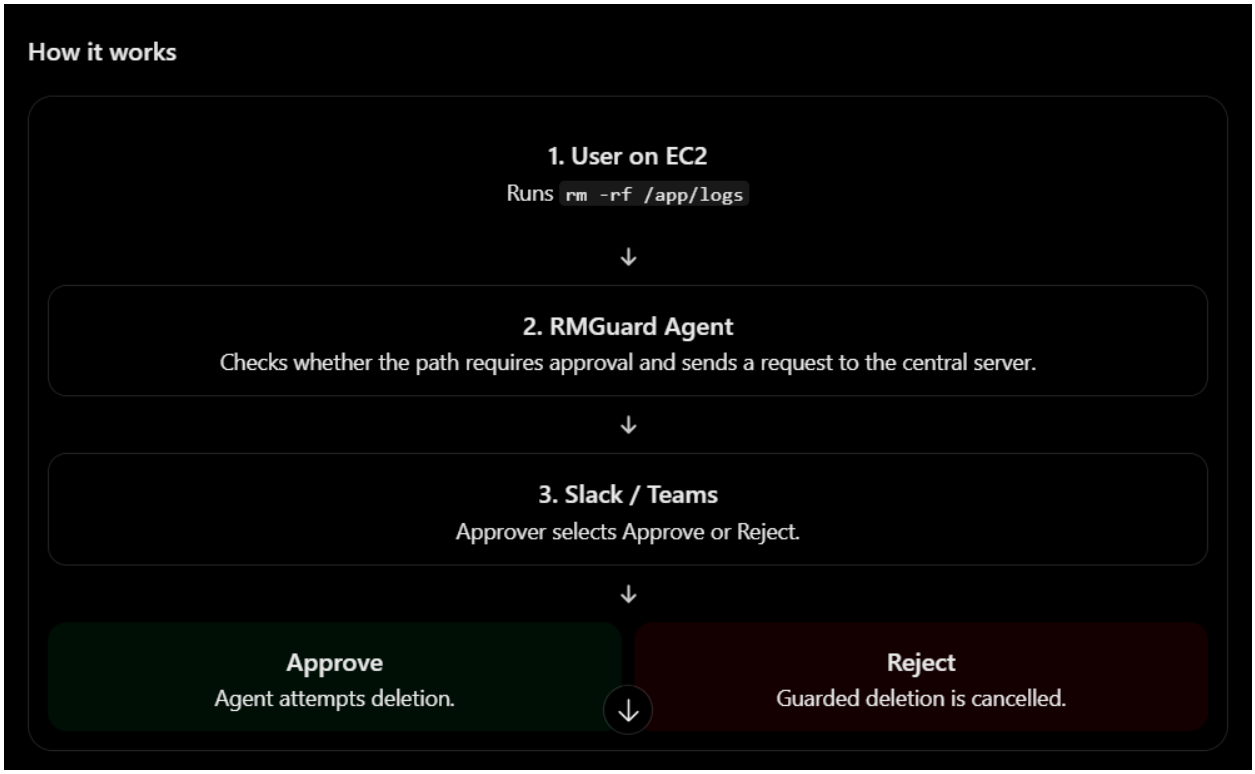
1. Intercepts supported deletion commands.
2. Identifies the user, instance, and target path.
3. Sends the request to the central RMGuard server.
4. Waits for the approval decision.
5. Attempts deletion only after approval and target validation.

Example: 100 EC2 instances

- Install the agent on all 100 instances.
- Use one central approval server.
- Receive deletion requests from any enrolled instance in Slack/Teams.
- Record approval decisions and execution results in the audit log.

Important: Your current code does not intercept every possible way to delete files. Install and test it carefully before relying on it in production.

How it works


1. User on EC2
Runs rm -rf /app/logs
2. RMGuard Agent
Checks whether the path requires approval and sends a request to the central server.
3. Slack / Teams
Approver selects Approve or Reject.
Approve
Agent attempts deletion.
Reject
Guarded deletion is cancelled.